Pedro Henrique Faluba Martins

Localizador de Região de Faltas de Alta Impedância como *Firmware* em Microcontrolador Simulado

Belo Horizonte

2026

Pedro Henrique Faluba Martins

Localizador de Região de Faltas de Alta Impedância como *Firmware* em Microcontrolador Simulado

Universidade Federal de Minas Gerais – UFMG

Escola de Engenharia

Curso de Graduação em Engenharia Elétrica

Belo Horizonte

2026

Sumário

1 INTRODUÇÃO 3

2 REVISÃO BIBLIOGRÁFICA 5
2.1 Aprendizagem de máquina em microcontroladores 5
2.2 Plataforma Cortex-M4F e CMSIS-DSP 6
2.3 Simulação funcional de microcontroladores 6
2.4 Custo computacional na localização de FAIs 6
2.5 Conclusão 7

3 METODOLOGIA 8
3.1 Dados e conjuntos de atributos 8
3.2 Modelos de classificação 10
3.3 Plataforma e implementação da extração 10
3.4 Verificação da equivalência 11
3.5 Método de medição 12
3.6 Varredura de configurações 13
3.7 Conclusões 13

4 RESULTADOS E DISCUSSÃO 14
4.1 Equivalência entre as implementações 14
4.2 Custo da configuração de referência 15
4.3 Efeito do conjunto de atributos 16
4.4 Complexidade do modelo e otimização de compilação 18
4.5 Adequação a dispositivos embarcados 19
4.6 Conclusões 19

5 CONCLUSÃO 20
5.1 Trabalhos futuros 21

REFERÊNCIAS 22

1 Introdução

As Faltas de Alta Impedância (FAIs) resultam do contato de um condutor energizado com materiais de condutividade muito reduzida e produzem correntes de baixa amplitude, aleatórias e não lineares, o que as torna de difícil detecção e localização pelos sistemas de proteção convencionais (BAHAROZU; ILHAN; SOYKAN, 2023). Em trabalho anterior (MARTINS, 2026), propôs-se um procedimento sistemático de seleção de atributos para a localização da região de uma FAI, obtidos exclusivamente a partir das medições da subestação. O resultado foi um conjunto de quinze famílias de atributos, extraídas pelas transformadas discreta de Fourier, de Stockwell e *wavelet* sobre uma janela de 20 ciclos, que superou com folga conjuntos aleatórios de mesma cardinalidade sob validação cruzada aninhada.

Naquele trabalho, argumentou-se ainda que os atributos selecionados não demandam elevado esforço computacional, visto que a extração completa dos 7056 atributos de uma simulação levou em torno de 0,2 s em um processador de uso geral, tempo inferior à própria duração da janela analisada. Esse argumento, porém, foi construído sobre um computador pessoal, enquanto os equipamentos de proteção e monitoramento instalados em subestações são, tipicamente, dispositivos embarcados, baseados em microcontroladores com memória da ordem de centenas de kilobytes e sem sistema operacional. A execução da localização no próprio dispositivo de medição dispensaria a comunicação com um centro de controle e a latência associada a ela, mas não se sabe se as transformadas empregadas, em especial a de Stockwell e a *wavelet*, cabem nos recursos de um dispositivo desse tipo, nem se a seleção de atributos se traduz em economia de recursos em relação ao uso de todos os atributos candidatos.

Além disso, o trabalho anterior entregou um conjunto de atributos, e não um modelo treinado, deixando como trabalho futuro o desenvolvimento de um algoritmo inteligente alimentado por esse conjunto. A escolha desse algoritmo também tem consequências sobre o dispositivo, pois famílias distintas de modelos ocupam memória e demandam processamento em proporções muito diferentes.

Nesse sentido, o objetivo do presente trabalho é implementar o localizador completo, isto é, a extração dos atributos selecionados e um modelo de classificação a jusante, como *firmware* em linguagem C para um microcontrolador ARM Cortex-M4F simulado, e medir a memória e o custo computacional demandados em diferentes configurações. Para isso, verifica-se primeiramente que a implementação embarcada reproduz a implementação de referência em *Python*, e então comparam-se o conjunto selecionado, todos os atributos candidatos, subconjuntos aleatórios e os primeiros atributos do ranqueamento,

combinados a quatro famílias de modelos, a diferentes complexidades de modelo e a dois níveis de otimização de compilação.

2 Revisão Bibliográfica

Este capítulo apresenta os conceitos e a literatura necessários para a implementação do localizador em um microcontrolador. Primeiramente, é apresentada a aprendizagem de máquina em microcontroladores e as ferramentas disponíveis para tal. Após, são descritas a arquitetura e as bibliotecas da plataforma adotada, bem como a simulação funcional empregada para executá-la. Por fim, são analisados os trabalhos de localização de FAIs baseados em algoritmos inteligentes quanto ao tratamento dado ao custo computacional.

2.1 Aprendizagem de máquina em microcontroladores

A execução de modelos de aprendizagem de máquina em microcontroladores, denominada *TinyML*, refere-se à inferência em dispositivos com memória da ordem de kilobytes e consumo da ordem de miliwatts, sem sistema operacional e sem comunicação obrigatória com servidores (WARDEN; SITUNAYAKE, 2019). Segundo Ray (2022), a área é motivada justamente pela redução da latência, do consumo de energia e da dependência de conectividade, ao custo de restrições severas de memória e de processamento, que exigem modelos e implementações dedicados.

Dentre as ferramentas disponíveis, destacam-se:

- **TensorFlow Lite Micro:** interpretador de modelos de redes neurais treinados no *TensorFlow*, voltado a microcontroladores e com suporte à quantização em 8 bits, cuja avaliação mostra baixa demanda de recursos e pequeno custo adicional de execução (DAVID et al., 2021).
- **CMSIS-NN:** biblioteca de núcleos de redes neurais otimizados para processadores Cortex-M, em ponto fixo de 8 e 16 bits, que reporta ganho de 4,6 vezes em tempo de execução sobre implementações de referência (LAI; SUDA; CHANDRA, 2018).
- **emlearn:** biblioteca que converte modelos treinados no *scikit-learn* (PEDREGOSA et al., 2011), como árvores de decisão, florestas aleatórias e redes neurais, em código C sem alocação dinâmica de memória (NORDBY, 2019). Por abranger os modelos clássicos de aprendizagem de máquina, e não apenas redes neurais, é a ferramenta adotada neste trabalho.

Quanto à avaliação, o *MLPerf Tiny* estabelece um conjunto padronizado de tarefas e métricas, a saber, acurácia, latência e energia, medidas em dispositivos reais (BANBURY et al., 2021). Suas tarefas, porém, são de reconhecimento de palavras-chave, detecção

visual, classificação de imagens e detecção de anomalias, nas quais a entrada do modelo é o próprio sinal ou um espectrograma simples. Problemas como o deste trabalho, em que a extração dos atributos envolve transformadas tempo-frequência sobre múltiplos canais, não estão representados.

2.2 Plataforma Cortex-M4F e CMSIS-DSP

Os processadores Cortex-M4 implementam a arquitetura Armv7E-M de 32 bits, com *pipeline* de três estágios, instruções dedicadas ao processamento de sinais e unidade de ponto flutuante opcional de precisão simples (ARM, 2026b). O microcontrolador STM32F407 integra esse processador, na versão com unidade de ponto flutuante, a 1 MB de memória *flash* e 192 KB de memória RAM, dos quais 64 KB constituem a memória acoplada ao núcleo (CCM), acessível apenas pelo processador e não pelos controladores de acesso direto à memória (STMICROELECTRONICS, 2026). Trata-se de um microcontrolador de uso difundido e de custo reduzido, o que o torna representativo de um dispositivo de medição embarcado.

Para o processamento de sinais, a biblioteca CMSIS-DSP oferece rotinas otimizadas para a família Cortex-M, dentre as quais a transformada rápida de Fourier (TRF) de sinais reais (ARM, 2026a). Essa transformada, contudo, é disponibilizada apenas para comprimentos que são potências de dois, de 32 a 4096 pontos, restrição relevante para a transformada de Stockwell, conforme discutido no Capítulo 3.

2.3 Simulação funcional de microcontroladores

O Renode é um simulador que executa o binário compilado para o microcontrolador sem modificações, com modelos dos periféricos de cada plataforma, como as interfaces seriais, e permite automatizar testes por meio do *Robot Framework* (ANTMICRO, 2026). O simulador contabiliza as instruções executadas pelo processador, mas é funcional e não preciso em ciclos, isto é, reproduz o comportamento do programa sem modelar o *pipeline*, os estados de espera da memória *flash* ou as latências das instruções. Com isso, o número de instruções executadas é adequado à comparação entre configurações, mas não corresponde ao tempo de execução em um dispositivo real.

2.4 Custo computacional na localização de FAIs

A literatura de localização de FAIs baseada em algoritmos inteligentes emprega majoritariamente redes neurais artificiais (FARIAS et al., 2018; LEDESMA et al., 2020; SARANGI; SAHU; ROUT, 2023), além de máquinas de vetor de suporte (JOGA; SINHA;

MAHARANA, 2023), regressão por processos gaussianos (MOLOI; JORDAAN; HAMAM, 2018) e comparações entre diferentes algoritmos (LOPES et al., 2025; LOPES, 2024), dentre as quais Lopes (2024) opta por um conjunto de árvores de decisão. Esses trabalhos avaliam o desempenho da localização, mas o custo de sua execução é tratado de forma qualitativa. Farias et al. (2018), por exemplo, destacam a simplicidade e o baixo custo de implementação de seu método, que emprega apenas medições da subestação, sem quantificar a memória ou o processamento exigidos.

Na comparação realizada em Martins (2026), o processamento exigido pelos trabalhos voltados à região é igualmente descrito em termos das técnicas empregadas e dos pontos de medição necessários, e em nenhum deles foram identificadas medições de memória ou de processamento em um dispositivo embarcado. O próprio trabalho anterior mediu o tempo de extração apenas em um computador pessoal.

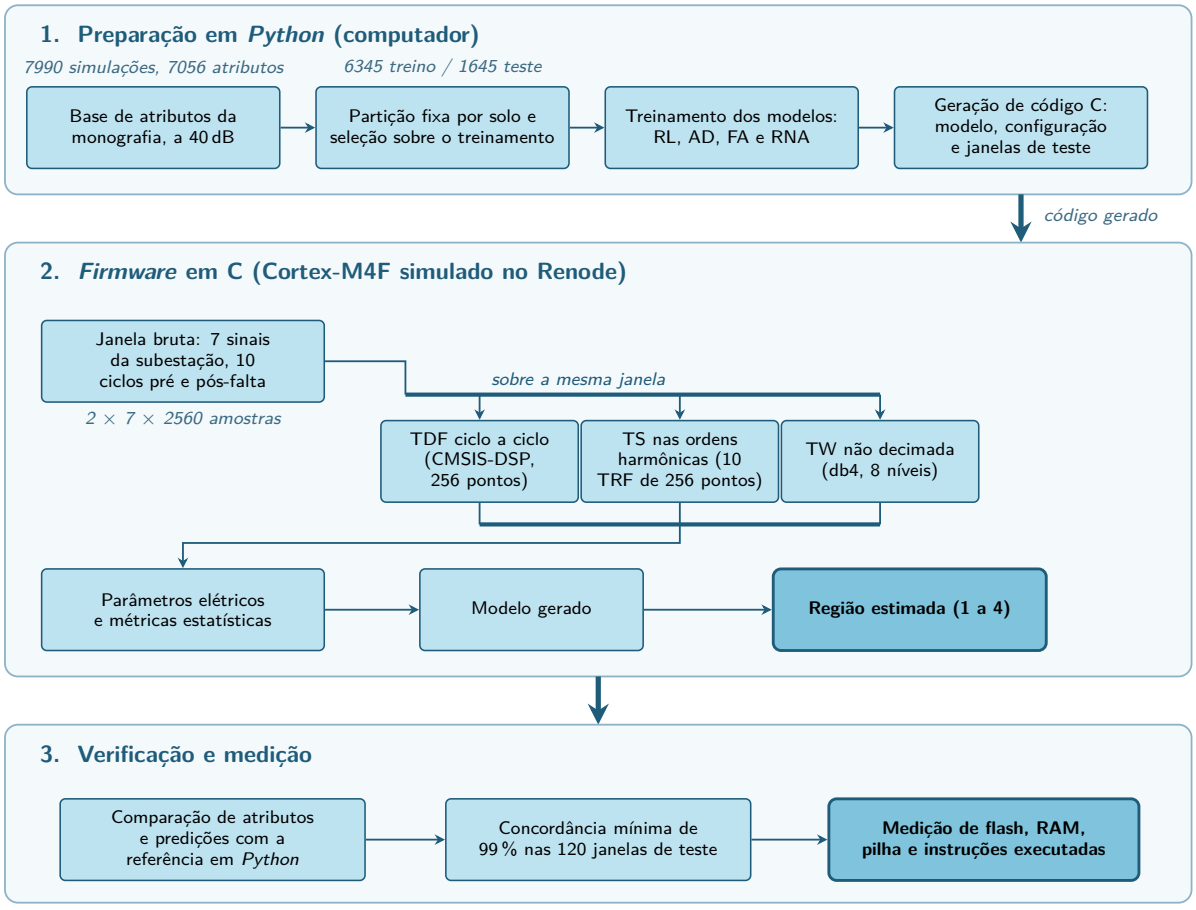
2.5 Conclusão

Este capítulo apresentou a aprendizagem de máquina em microcontroladores, a plataforma Cortex-M4F com a biblioteca CMSIS-DSP e a simulação funcional por meio do Renode. Duas observações delimitam a contribuição do presente trabalho. A primeira é que as ferramentas e os *benchmarks* de *TinyML* concentram-se no modelo, enquanto no problema da localização de FAIs a extração dos atributos envolve transformadas tempo-frequência potencialmente mais custosas que o próprio modelo. A segunda é que os trabalhos de localização de FAIs não quantificam a memória e o processamento demandados em um dispositivo embarcado. O presente trabalho mede ambos, sobre o conjunto de atributos obtido em Martins (2026).

3 Metodologia

Este capítulo apresenta a metodologia utilizada para implementar o localizador como *firmware* e medir os recursos demandados. Primeiramente são apresentados os dados, os conjuntos de atributos e os modelos avaliados, seguidos da plataforma e da implementação da extração em C. Por fim, são apresentados os procedimentos de verificação da equivalência entre as implementações, o método de medição e a varredura de configurações. A metodologia pode ser, em alto nível, resumida no fluxo apresentado na Figura 1.

Figura 1 – Fluxo de implementação e avaliação do localizador como *firmware*



Fonte: Elaborado pelo autor

3.1 Dados e conjuntos de atributos

Foi utilizada a base de atributos construída em Martins (2026), com 7056 atributos por simulação sobre 7990 simulações de FAIs no sistema teste IEEE de 34 barras adaptado,

ao nível de ruído de 40 dB adotado naquele trabalho. A partição entre treinamento e teste foi fixada uma única vez e mantida em todos os experimentos. Ela reserva para o teste sete dos 34 sinais de FAI, sorteados com semente fixa, o que resulta em 6345 simulações de treinamento e 1645 de teste com a mesma proporção de regiões. A partição por sinal de FAI, e não por simulação, impede que uma mesma forma de onda de arco figure simultaneamente no treinamento e no teste.

O ranqueamento publicado em Martins (2026) foi obtido sobre todas as simulações, incluindo, portanto, as que compõem o conjunto de teste deste trabalho. Por essa razão, a cadeia de seleção daquele trabalho, isto é, agrupamento por correlação, filtragem, etapa embutida, etapa *wrapper* e seleção por estabilidade, foi reexecutada sem alterações sobre as simulações de treinamento apenas, ao longo de cem execuções. O índice de estabilidade obtido foi de 0,256, abaixo dos 0,315 obtidos sobre o conjunto completo, o que é esperado diante do menor número de sinais de FAI em cada subamostra, e o conjunto resultante compartilha dez dos quinze agrupamentos com o publicado. Esse conjunto, apresentado na Tabela 1, é o conjunto de referência deste trabalho.

Tabela 1 – Conjunto de quinze atributos selecionado sobre as simulações de treinamento

#	π	**Técnica**	**Parâmetro**	**Canal**	**Ordem**	**Métrica**
1	0,95	TDF	Módulo da impedância	V_f / I_f	h1	máximo
2	0,76	TW	Módulo	V_f	D3	diferença
3	0,68	TS	DHT	I_f	todas	razão
4	0,48	TW	Módulo	I_n	D7	máximo
5	0,43	TW	Módulo da impedância	V_f / I_f	D6	razão
6	0,38	TW	Energia	I_f	D8	amplitude
7	0,35	TDF	Módulo da impedância	ΣV / I_n	h3	média
8	0,29	TS	Módulo da impedância	ΣV / I_n	h3	razão
9	0,25	TDF	Energia	V_f	h1	máximo
10	0,21	TS	Energia	V_f	h8	média
11	0,21	TS	Módulo	V_2	h12	diferença
12	0,20	TS	Ângulo	I_n	h1	média
13	0,19	TDF	Ângulo	V_f	h1	média
14	0,19	TS	Módulo	V_2	h10	diferença
15	0,15	TDF	Energia	V_2	h14	média

Fonte: Elaborado pelo autor. π: frequência de seleção nas cem execuções; TDF, TS e TW: transformadas discreta de Fourier, de Stockwell e *wavelet*; V_f e I_f: tensão e corrente da fase faltosa; I_n: corrente de neutro; ΣV: soma das tensões trifásicas; V_2: tensão de sequência negativa; hn: ordem harmônica; Dn: banda diádica da TW.

Além do conjunto de referência, foram avaliados o conjunto publicado em Martins (2026), mantido apenas como comparação por ter sido selecionado com acesso às simulações de teste, todos os 7056 atributos candidatos, os primeiros três, cinco e dez atributos do conjunto de referência e dez subconjuntos de quinze atributos sorteados uniformemente dentre os 7056, cada um com semente fixa.

3.2 Modelos de classificação

Foram avaliadas quatro famílias de modelos, todas treinadas com o *scikit-learn* (PEDREGOSA et al., 2011) e com ponderação das classes:

- **Regressão logística (RL):** com regularização L2, por ser o classificador de referência empregado na etapa *wrapper* de Martins (2026).
- **Árvore de decisão (AD):** com profundidade máxima de oito níveis.
- **Floresta aleatória (FA):** com trinta árvores de profundidade máxima oito, cuja predição é a média das proporções de classe das folhas.
- **Rede neural artificial (RNA):** perceptron com uma camada oculta de dezesseis neurônios e ativação ReLU.

A RL e a RNA recebem os atributos após a transformação logarítmica com sinal e a padronização adotadas em Martins (2026). Já a AD e a FA recebem os atributos sem transformação, visto que árvores de decisão são invariantes a transformações monotônicas, o que dispensa quinze cálculos de logaritmo por janela no dispositivo.

A conversão para C empregou o emlearn (NORDBY, 2019) para a AD e a FA. Para a RL, não suportada pela biblioteca, e para a RNA, foram gerados os coeficientes e uma rotina própria de inferência. No caso da RNA, a rotina do emlearn exige que os vetores intermediários tenham a largura da camada de entrada, o que, com os 7056 atributos, demandaria 55 KB adicionais de RAM.

3.3 Plataforma e implementação da extração

O *firmware* foi desenvolvido para o STM32F407, simulado no Renode a partir da plataforma da placa de desenvolvimento STM32F4 Discovery, e compilado com otimização para tamanho, unidade de ponto flutuante de precisão simples por *hardware* e eliminação de código não utilizado. Todas as variáveis são estáticas, sem alocação dinâmica de memória. O modelo de plataforma do simulador dispõe de mais memória que o microcontrolador real e não contém a memória CCM. Por isso, a CCM foi acrescentada ao modelo, e os limites reais de 1 MB de *flash* e 128 KB de RAM principal foram impostos na ligação do programa.

A extração em C é genérica, isto é, capaz de calcular qualquer um dos 7056 atributos. O conjunto a ser calculado é descrito por uma tabela gerada automaticamente a partir dos atributos do modelo, que indica, para cada transformada, os canais e as ordens

harmônicas ou bandas necessárias. Somente essas combinações são calculadas, e os vetores de trabalho são dimensionados por elas. As três transformadas foram implementadas como segue:

- **TDF:** aplicada ciclo a ciclo por meio da TRF de sinais reais da CMSIS-DSP, com 256 pontos.
- **TS:** em Martins (2026), cada voz harmônica é obtida por uma transformada inversa de 2560 pontos, comprimento de dez ciclos que não é potência de dois. A transformada direta de 2560 pontos foi obtida por decimação, a partir de dez TRFs de 256 pontos. Como apenas a média de cada voz em cada ciclo é utilizada, a transformada inversa foi substituída por uma forma fechada, dada pela Equação (3.1), em que H é a transformada direta, $k_0 = 10h$ é a raia do harmônico h, G_h é a janela gaussiana da transformada (STOCKWELL; MANSINHA; LOWE, 1996), B é a média de um termo exponencial sobre as 256 amostras de um ciclo e c é o ciclo. A soma foi truncada em $|m| \leq 1{,}5\,k_0$, onde a janela gaussiana vale e^{-44}, contribuição desprezível em precisão simples.

$$X_{h,c} = \frac{2}{N} \sum_{|m| \leq 15h} H[k_0 + m]\, G_h[m]\, B[m]\, e^{j2\pi mc/10} \tag{3.1}$$

- **TW:** implementada na forma não decimada, por convolução circular com os filtros da Daubechies de quarta ordem sobreamostrados a cada nível, com o valor eficaz de cada ciclo acumulado durante a própria filtragem, sem armazenar as bandas.

As componentes de sequência negativa da TW, que demandam a rotação de fase pela transformada de Hilbert, empregam a transformada inversa de 2560 pontos obtida pela mesma decimação. Todos os cálculos são realizados em precisão simples.

3.4 Verificação da equivalência

A implementação de referência foi reescrita em *Python*, sem dependência do código de Martins (2026), e reproduz a matriz de atributos daquele trabalho, em todas as 1645 simulações de teste e todos os 7056 atributos, com erro relativo máximo de $6{,}9 \times 10^{-8}$, correspondente ao arredondamento do armazenamento em precisão simples. Cada rotina em C possui uma contraparte nessa referência, e os algoritmos reformulados, isto é, a decimação e a forma fechada da TS, foram verificados contra as definições com erro da ordem do arredondamento em precisão dupla.

A referência é calculada sobre a mesma janela em precisão simples recebida pelo dispositivo, de modo que apenas a aritmética da implementação em C difere entre as duas. Um atributo calculado em C, c, é considerado equivalente ao de referência, p, quando

$|c-p| \leq 10^{-3}|p|+10^{-3}\sigma$, em que σ é o desvio padrão do atributo no conjunto de teste, sendo os ângulos comparados por diferença envolvente. O termo proporcional a σ é necessário porque atributos de diferença, de curtose e angulares assumem valores próximos de zero, nos quais o erro relativo isolado é instável.

Cada janela bruta ocupa 140 KB, de modo que apenas seis ou sete janelas cabem na *flash* ao lado do programa. Foram selecionadas 120 janelas do conjunto de teste, estratificadas por região, executadas em lotes, cada qual compilado em um *firmware* distinto. Uma configuração foi considerada válida quando a predição do dispositivo concorda com a do *scikit-learn* em ao menos 99% dessas janelas.

3.5 Método de medição

As métricas adotadas, bem como a forma de obtenção de cada uma, foram:

- **Memória *flash*:** soma das seções de código e de dados constantes, obtida a partir do mapa de ligação e separada por módulo, isto é, extração, CMSIS-DSP, modelo, biblioteca padrão e aplicação, excluídas as janelas de teste.
- **RAM estática:** soma das variáveis inicializadas e não inicializadas, incluindo a CCM, obtida da mesma forma.
- **Pilha:** pico de utilização, medido pelo preenchimento prévio da região da pilha com um padrão conhecido e pela verificação, ao final da execução, da porção sobrescrita.
- **Instruções executadas:** lidas do contador do simulador por uma rotina associada ao endereço de uma função de marcação, executada no instante exato em que o processador a alcança. A extração foi medida como a média sobre as janelas do lote, executadas duas vezes, e a inferência como a média sobre os atributos de referência de até 1645 simulações de teste, visto que o custo de uma árvore depende do caminho percorrido. Uma janela adicional foi medida com marcações entre as etapas da extração.

A precisão da leitura do contador foi verificada sobre um laço de comprimento conhecido. A alternativa de pausar a simulação ao surgir uma mensagem na interface serial mostrou-se imprecisa, pois o simulador interrompe a execução apenas em seu próximo ponto de sincronização, até milhões de instruções depois.

Vale ressaltar que o número de instruções não equivale ao tempo de execução em um dispositivo real. Em um STM32F407 a 168 MHz, o tempo depende ainda do esvaziamento do *pipeline* a cada desvio, dos estados de espera da *flash*, apenas parcialmente ocultados pelo seu acelerador, e das latências das instruções, que vão de um ciclo para

operações inteiras simples a quatorze ciclos para a divisão e a raiz quadrada em ponto flutuante. Por isso, as estimativas de tempo apresentadas assumem uma instrução por ciclo e são tratadas apenas como estimativas grosseiras.

3.6 Varredura de configurações

A varredura avaliou 80 configurações, organizadas em três grupos, cada um variando um fator por vez:

- **Conjunto de atributos e modelo:** os dezesseis conjuntos da Seção 3.1, combinados às quatro famílias de modelos, com otimização para tamanho, totalizando 64 configurações.
- **Complexidade do modelo:** sobre o conjunto de referência, FA com 5, 10 e 30 árvores e profundidades de 4, 8 e 12, AD com profundidades de 4 e 12 e RNA com 8 e 32 neurônios.
- **Otimização de compilação:** as quatro famílias de modelos sobre o conjunto de referência, compiladas com otimização para velocidade.

Cada configuração é treinada, convertida, executada no dispositivo simulado sobre as 120 janelas e medida de forma automatizada. O comprimento da janela não foi variado, visto que a amostragem de 256 amostras por ciclo e a janela de 20 ciclos integram a própria definição dos atributos de Martins (2026).

3.7 Conclusões

Este capítulo apresentou a metodologia empregada para implementar e avaliar o localizador como *firmware*. O conjunto de referência foi obtido pela mesma cadeia de seleção de Martins (2026), executada apenas sobre as simulações de treinamento, e foi combinado a quatro famílias de modelos. A extração foi implementada em C sobre a CMSIS-DSP, com as transformadas reformuladas para comprimentos e operações compatíveis com o microcontrolador, e a equivalência com a referência em *Python* é verificada em cada configuração antes de qualquer medição. O capítulo seguinte apresenta os resultados obtidos.

4 Resultados e Discussão

Este capítulo apresenta os resultados obtidos de cada etapa da metodologia, iniciando pela equivalência entre as implementações, seguida do custo da configuração de referência e dos efeitos do conjunto de atributos, da complexidade do modelo e da otimização de compilação.

4.1 Equivalência entre as implementações

As quatro famílias de modelos, convertidas para C e alimentadas pelo conjunto de referência, concordaram com o *scikit-learn* em todas as 1645 simulações de teste, tanto compiladas para o computador quanto executadas no dispositivo simulado. Ao longo da varredura, apenas oito das 25 configurações com FA divergiram, em uma a quatro das 1645 simulações, sempre de forma idêntica no computador e no dispositivo. A divergência decorre da representação em 8 bits das proporções de classe das folhas pelo emlearn, que inverte a decisão quando duas regiões estão praticamente empatadas, e não de um erro da implementação.

Os quinze atributos do conjunto de referência calculados no dispositivo atenderam ao critério de equivalência em todos os 1800 valores das 120 janelas, com o maior desvio correspondendo a 11,5% da tolerância. Esse desvio ocorre no módulo da impedância entre a soma das tensões e a corrente de neutro, já que a soma das três tensões de fase, de aproximadamente 21 kV, cancela-se em um resíduo pequeno, cuja resolução em precisão simples é limitada.

Estendida aos 7056 atributos, a verificação atendeu ao critério em 99,99% dos valores. Os 244 atributos que falharam em ao menos uma janela concentram-se em impedâncias entre grandezas que se cancelam, como a soma das tensões e as componentes de sequência negativa, e em momentos estatísticos de séries quase constantes. Nesses atributos, o erro da aritmética em precisão simples é, em mediana, cerca de sete vezes maior que o produzido apenas pelo armazenamento da janela em precisão simples, o que constitui um limite da representação numérica, e não da implementação. Ainda assim, todas as 80 configurações da varredura atenderam ao critério de concordância das predições, com mínimo de 99,17%, correspondente a uma divergência em 120 janelas, ocorrida em uma das FAs mencionadas acima.

4.2 Custo da configuração de referência

A Tabela 2 apresenta os recursos demandados pelo conjunto de referência com cada família de modelos.

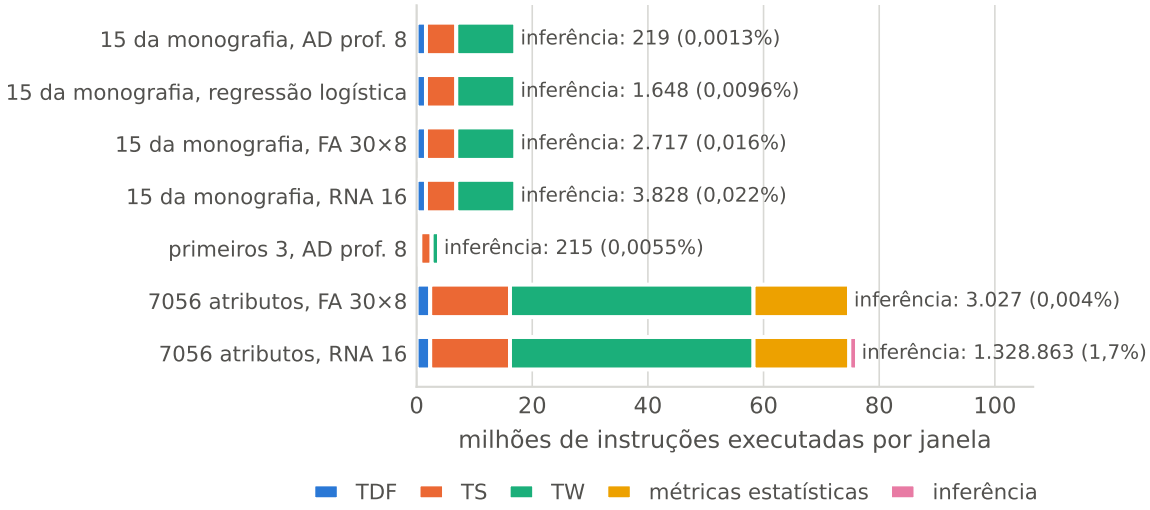
Tabela 2 – Recursos demandados pelo conjunto de referência

	RL	**AD**	**FA**	**RNA**
macro-F1	0,680	0,741	0,781	0,689
Flash total (KB)	18,8	21,9	131,5	19,9
Flash do modelo (KB)	0,6	3,7	113,2	1,7
RAM estática (KB)	97,3	97,3	97,3	97,5
Pico de pilha (B)	1800	1800	1800	1800
Instruções por extração (milhões)	17,17	17,17	17,17	17,17
Instruções por inferência	1648	219	2717	3828

Fonte: Elaborado pelo autor. macro-F1 sobre as 1645 simulações de teste.

A extração dos atributos demanda 17,17 milhões de instruções por janela, enquanto a inferência demanda de 219 a 3828 instruções, isto é, cerca de 0,02% do total no pior caso. O custo do localizador é, portanto, determinado pela extração, e não pelo modelo. Mesmo na RL, o modelo mais simples em memória, a maior parte das instruções de inferência corresponde aos quinze cálculos de logaritmo da transformação de entrada, e não à combinação linear em si. A Figura 2 detalha essa distribuição por etapa da extração.

Figura 2 – Instruções executadas por janela, por etapa da extração e na inferência



Fonte: Elaborado pelo autor

No conjunto de referência, a TW responde por 60% das instruções da extração, a TS por 31%, a TDF por 9% e as métricas estatísticas por apenas 0,2%. O peso da

TW decorre da forma não decimada, em que cada um dos até oito níveis filtra todas as 2560 amostras de cada janela. A TS, cuja definição exige uma transformada inversa por harmônico, responde por menos de um terço do total por meio da forma fechada da Equação (3.1), mesmo sendo a DHT calculada sobre as vinte ordens harmônicas. Vale ressaltar que a própria implementação influencia esses números, já que a remoção do tratamento circular de índices no interior da filtragem da TW reduziu as instruções dessa etapa em 32%, com resultados idênticos.

Quanto à memória, a *flash* do conjunto de referência é dominada pelo modelo no caso da FA, de 113,2 KB, enquanto a extração ocupa 4,0 KB, a CMSIS-DSP 5,8 KB e a biblioteca padrão 7,3 KB. Já a RAM estática, de aproximadamente 97 KB em todas as famílias, corresponde quase integralmente aos vetores de trabalho da extração, isto é, os coeficientes das transformadas de cada canal, as tabelas gaussianas da TS e os vetores intermediários da TW. Em uma estimativa grosseira, à razão de uma instrução por ciclo a 168 MHz, a extração levaria cerca de 102 ms, abaixo dos aproximadamente 333 ms de sinal contidos nas duas janelas.

4.3 Efeito do conjunto de atributos

A Tabela 3 apresenta o desempenho de cada conjunto de atributos com cada família de modelos, juntamente com as instruções de extração, e a Figura 3 relaciona o desempenho ao custo computacional de todas as configurações.

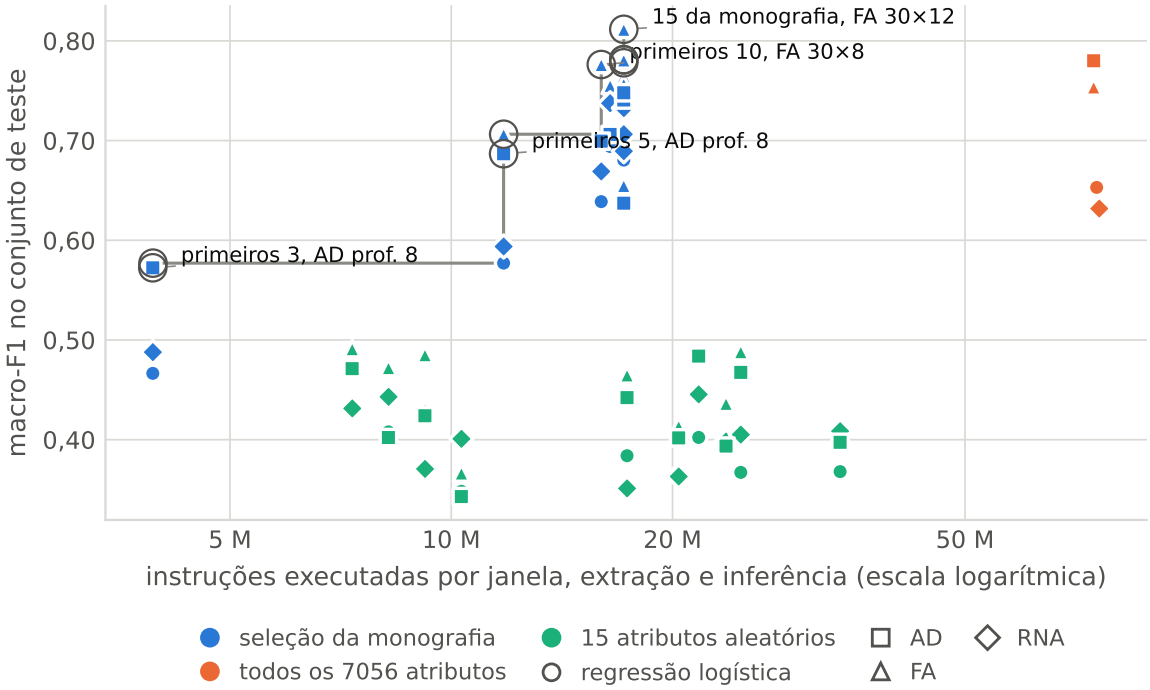
Tabela 3 – Desempenho e custo de extração por conjunto de atributos

Conjunto	**RL**	**AD**	**FA**	**RNA**	**Instruções (milhões)**
Referência (15)	0,680	0,741	0,781	0,689	17,2
Publicado (15)	0,694	0,706	0,755	0,737	16,4
Primeiros 10	0,639	0,699	0,776	0,669	16,0
Primeiros 5	0,577	0,687	0,706	0,594	11,8
Primeiros 3	0,467	0,572	0,577	0,488	3,9
Aleatórios (15), média	0,389	0,423	0,452	0,402	18,9
Aleatórios (15), máximo	0,429	0,484	0,491	0,445	33,8
Todos (7056)	0,653	0,780	0,754	0,632	74,8

Fonte: Elaborado pelo autor. Valores de macro-F1 sobre as 1645 simulações de teste. Instruções de extração por janela; para os conjuntos aleatórios, mediana e máximo dos dez sorteios.

O conjunto de referência superou todos os dez conjuntos aleatórios em todas as famílias de modelos, visto que mesmo o seu pior resultado, de 0,680 com a RL, supera o melhor resultado aleatório, de 0,491 com a FA. Esse resultado é ainda mais relevante

Figura 3 – Desempenho em função das instruções executadas por janela



Fonte: Elaborado pelo autor. Os pontos circundados formam a fronteira de configurações não superadas simultaneamente em desempenho e custo.

quando se observa que os conjuntos aleatórios demandam entre 7,3 e 33,8 milhões de instruções, com mediana de 18,9 milhões, isto é, custo comparável ao do conjunto de referência. A seleção, portanto, não reduz o custo em relação a um conjunto arbitrário de mesma cardinalidade, mas concentra nele a informação sobre a região.

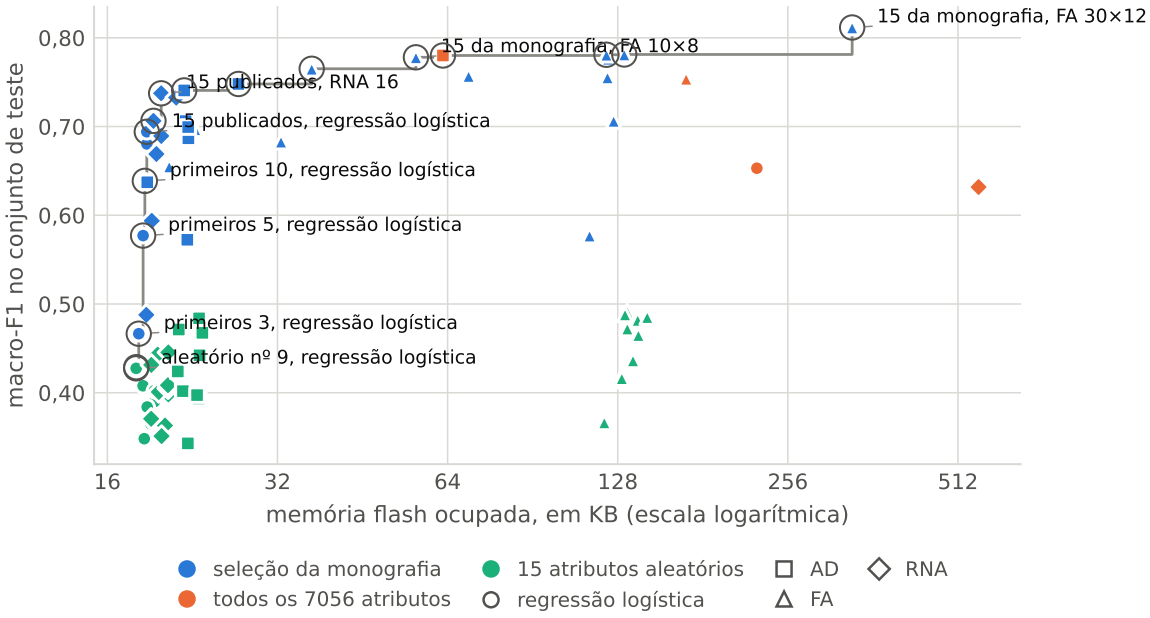
Em relação aos 7056 atributos candidatos, o conjunto de referência demanda 4,4 vezes menos instruções e não perde desempenho. A FA alcança 0,781 com o conjunto de referência e 0,754 com todos os atributos, enquanto a AD alcança 0,741 e 0,780, respectivamente, a única família em que o conjunto completo é superior. Além disso, o conjunto completo exige de 139 a 167 KB de RAM estática, que só cabem no microcontrolador com o uso da CCM, e de 63 a 557 KB de *flash*, conforme o modelo. Na extração dos 7056 atributos, as métricas estatísticas passam a responder por 22% das instruções, contra 0,2% no conjunto de referência.

Os primeiros atributos do ranqueamento mostram que o custo pode ser reduzido com perda gradual de desempenho. Os dez primeiros alcançam 0,776 com a FA, praticamente o mesmo resultado do conjunto completo de quinze, com 16,0 milhões de instruções, enquanto os cinco primeiros alcançam 0,706 com 11,8 milhões e os três primeiros, 0,577 com 3,9 milhões. A redução é mais acentuada nos três primeiros porque estes dispensam a maior parte das bandas da TW e das vozes da TS. Já o conjunto publicado obteve desempenho próximo ao de referência, superior com a RL e a RNA e inferior com a AD e

a FA, sendo que essa comparação é otimista para o conjunto publicado, selecionado com acesso às simulações de teste.

A Figura 4 relaciona o mesmo desempenho à memória *flash*. Diferentemente das instruções, a *flash* pouco varia com o conjunto de atributos, situando-se entre 18 e 22 KB para todos os conjuntos de quinze atributos, exceto com a FA. Isso ocorre porque o código de extração é o mesmo para qualquer conjunto, variando apenas a tabela que o configura. Com isso, a *flash* é determinada pelo modelo, enquanto a seleção de atributos determina as instruções executadas e a RAM.

Figura 4 – Desempenho em função da memória *flash* ocupada



Fonte: Elaborado pelo autor

4.4 Complexidade do modelo e otimização de compilação

Na FA, o aumento de 5 para 30 árvores e de profundidade 4 para 12 elevou o macro-F1 de 0,655 para 0,812, o melhor resultado da varredura, ao custo de 314 KB de *flash* para o modelo, contra 2,4 KB da menor configuração. Os ganhos, contudo, não são uniformes. Com dez árvores de profundidade oito, a FA alcança 0,778 com 38 KB, enquanto as trinta árvores de mesma profundidade acrescentam apenas 0,003 ao custo de 75 KB adicionais. Na AD, a profundidade 4 resultou em 0,637 e a profundidade 12 em 0,748, próxima dos 0,741 da profundidade 8. Na RNA, oito, dezesseis e trinta e dois neurônios resultaram em 0,706, 0,689 e 0,733, sem tendência monotônica. Em todos os casos, a inferência permaneceu abaixo de 6300 instruções, de modo que a complexidade do modelo afeta essencialmente a *flash*.

A otimização para velocidade elevou as instruções de extração em 3,2%, de 17,17 para 17,71 milhões, reduziu as instruções de inferência da AD em 31% e acrescentou de 1,3 a 9,3 KB de *flash*. O aumento das instruções de extração não significa que o código seja mais lento, já que esse nível de otimização reorganiza o código para reduzir ciclos por meio de desvios e acessos à memória, efeito que o simulador funcional não captura. Esse resultado ilustra a limitação discutida na Seção 3.5.

4.5 Adequação a dispositivos embarcados

O *firmware* de todas as 80 configurações coube no STM32F407, com no máximo 557 KB de *flash* e 169 KB de RAM estática somada à pilha, sendo a CCM necessária apenas com os 7056 atributos. Nenhuma configuração caberia, contudo, em um microcontrolador de 32 KB de RAM, típico da classe Cortex-M0+, visto que a menor RAM estática observada foi de 49 KB.

Há, porém, um requisito que não está contabilizado nesses valores. Neste trabalho, as janelas brutas residem na *flash* como dados de teste, mas um dispositivo real precisaria mantê-las em RAM, à medida que fossem adquiridas. As duas janelas dos sete sinais ocupam de 120 a 140 KB em precisão simples, conforme os canais exigidos pelo conjunto, valor que, somado aos vetores de trabalho da extração, excede a RAM do microcontrolador adotado.

4.6 Conclusões

Este capítulo apresentou os resultados obtidos. A implementação em C reproduziu a referência em *Python* em todas as configurações avaliadas, com a única ressalva de atributos cujo cálculo é sensível à precisão simples. O custo do localizador é determinado pela extração dos atributos, que demanda cerca de 17 milhões de instruções por janela no conjunto de referência, contra no máximo alguns milhares na inferência. O conjunto de referência superou todos os conjuntos aleatórios de mesma cardinalidade, a custo comparável, e superou o conjunto completo de atributos em três das quatro famílias, a menos de um quarto do custo. Por fim, a *flash* é determinada pelo modelo e a RAM pela extração, e o armazenamento da janela bruta constitui o principal obstáculo a uma implementação em um dispositivo real.

5 Conclusão

Este trabalho implementou o localizador de região de FAIs como *firmware* em linguagem C para um microcontrolador ARM Cortex-M4F simulado, a partir do conjunto de atributos obtido em Martins (2026), e mediu a memória e o custo computacional demandados em 80 configurações. Para isso, a cadeia de seleção daquele trabalho foi reexecutada apenas sobre as simulações de treinamento, a fim de que o conjunto de teste permanecesse inédito, e o conjunto resultante foi combinado à regressão logística, à árvore de decisão, à floresta aleatória e a uma rede neural artificial.

A extração foi implementada sobre a biblioteca CMSIS-DSP, cuja transformada rápida de Fourier se restringe a comprimentos que são potências de dois. A transformada de Stockwell, definida sobre uma janela de 2560 amostras, foi obtida por decimação e por uma forma fechada para a média de cada voz em cada ciclo, que dispensa as transformadas inversas, e a transformada *wavelet* não decimada acumula o valor eficaz de cada ciclo durante a filtragem. Em todas as configurações, as predições do dispositivo concordaram com as da implementação de referência em ao menos 99% das janelas de teste, e os atributos do conjunto de referência foram reproduzidos dentro da tolerância em todas as janelas.

Os resultados mostram que o custo do localizador está na extração dos atributos e não no modelo. No conjunto de referência, a extração demanda 17,17 milhões de instruções por janela, das quais 60% na transformada *wavelet*, enquanto a inferência demanda no máximo 0,02% do total. A seleção de atributos reduziu esse custo em 4,4 vezes em relação aos 7056 atributos candidatos sem perda de desempenho na floresta aleatória, que alcançou macro-F1 de 0,781, e superou todos os conjuntos aleatórios de mesma cardinalidade, cujo custo é comparável. Na floresta aleatória, os dez primeiros atributos do ranqueamento mantiveram praticamente o mesmo desempenho com custo ligeiramente menor. Já a escolha do modelo determina a memória *flash*, de 0,6 KB na regressão logística a 314 KB na maior floresta avaliada, que obteve o melhor resultado, de 0,812. Com isso, o argumento de baixo custo computacional apresentado em Martins (2026) se confirma em um microcontrolador, com a ressalva de que a extração, e não o modelo, deve ser o foco de qualquer otimização.

Algumas limitações, porém, devem ser consideradas. O simulador é funcional e, portanto, as instruções executadas permitem comparar configurações, mas não determinam o tempo de execução, que depende dos estados de espera da memória e das latências das instruções no dispositivo real. Além disso, as janelas brutas residem na *flash* como dados de teste, ao passo que um dispositivo real precisaria armazená-las em RAM, o que

excede a capacidade do microcontrolador adotado. Por fim, os sinais provêm de simulações, a fase faltosa é obtida dos metadados, conforme já ocorria em Martins (2026), e o consumo de energia não foi medido.

5.1 Trabalhos futuros

Os resultados obtidos abrem diferentes caminhos para a continuidade deste trabalho, dentre os quais se pode destacar:

- **Execução em um dispositivo real:** a execução do mesmo *firmware* em uma placa STM32F4, com a leitura do contador de ciclos do processador e a medição do consumo de energia, permitiria converter as instruções aqui medidas em tempo e energia, e verificar o atendimento ao requisito de tempo real.
- **Aquisição contínua dos sinais:** a substituição das janelas armazenadas por uma aquisição contínua, com conversores analógico-digitais e transferência direta para a memória, e o processamento dos ciclos à medida que são adquiridos reduziriam a memória necessária para a janela bruta, principal obstáculo identificado.
- **Redução do custo da extração:** como a extração domina o custo, a otimização da transformada *wavelet*, a redução dos vetores de trabalho às ordens efetivamente utilizadas e o uso de aritmética em ponto fixo poderiam reduzir instruções e RAM, desde que a equivalência com a referência seja preservada.
- **Comparação com redes neurais quantizadas:** a comparação com modelos executados pelo TensorFlow Lite Micro (DAVID et al., 2021) em 8 bits permitiria avaliar o compromisso entre precisão e memória, ainda que a inferência represente uma parcela pequena do custo total.
- **Inclusão do custo na seleção de atributos:** o custo de extração de cada atributo, aqui medido, poderia ser incorporado como critério da própria seleção, favorecendo, dentre atributos de desempenho equivalente, aqueles de menor custo no dispositivo.

Referências

ANTMICRO. *Renode documentation*. 2026. Disponível em: <https://renode.readthedocs.io>. Acesso em: 1 nov. 2026. Citado na página 6.

ARM. *CMSIS-DSP: Embedded compute library for Cortex-M and Cortex-A*. 2026. Disponível em: <https://github.com/ARM-software/CMSIS-DSP>. Acesso em: 1 nov. 2026. Citado na página 6.

ARM. *Cortex-M4 Technical Reference Manual*. 2026. Disponível em: <https://developer.arm.com/documentation/100166/0001>. Acesso em: 1 nov. 2026. Citado na página 6.

BAHAROZU, E.; ILHAN, S.; SOYKAN, G. High impedance fault localization: A comprehensive review. *Electric Power Systems Research*, v. 214, p. 108892, 2023. ISSN 0378-7796. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0378779622009439>. Citado na página 3.

BANBURY, C. et al. MLPerf tiny benchmark. In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. [s.n.], 2021. Disponível em: <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html>. Citado na página 5.

DAVID, R. et al. TensorFlow Lite Micro: Embedded machine learning for TinyML systems. In: *Proceedings of Machine Learning and Systems*. [s.n.], 2021. v. 3. Disponível em: <https://proceedings.mlsys.org/paper_files/paper/2021/hash/6c44dc73014d66ba49b28d483a8f8b0d-Abstract.html>. Citado 2 vezes nas páginas 5 e 21.

FARIAS, P. E. et al. Non-linear high impedance fault distance estimation in power distribution systems: A continually online-trained neural network approach. *Electric Power Systems Research*, v. 157, p. 20–28, 2018. ISSN 0378-7796. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0378779617304698>. Citado 2 vezes nas páginas 6 e 7.

JOGA, S. R. K.; SINHA, P.; MAHARANA, M. K. A novel graph search and machine learning method to detect and locate high impedance fault zone in distribution system. *Engineering Reports*, v. 5, n. 1, p. e12556, 2023. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1002/eng2.12556>. Citado na página 7.

LAI, L.; SUDA, N.; CHANDRA, V. *CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs*. 2018. ArXiv:1801.06601. Disponível em: <https://arxiv.org/abs/1801.06601>. Citado na página 5.

LEDESMA, J. J. G. et al. A two-level ANN-based method using synchronized measurements to locate high-impedance fault in distribution systems. *Electric Power Systems Research*, v. 188, p. 106576, 2020. ISSN 0378-7796. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0378779620303801>. Citado na página 6.

LOPES, G. N. *Detecção e Localização de Faltas de Alta Impedância em Sistemas de Distribuição de Energia Elétrica com Geração Distribuída*. Tese (Doutorado) — Universidade de São Paulo, Escola de Engenharia de São Carlos, 2024. Citado na página 7.

LOPES, G. N. et al. High impedance fault location in distribution systems: A novel approach with enhanced metrics and intelligent algorithms. *IEEE Access*, v. 13, p. 108466–108480, 2025. Citado na página 7.

MARTINS, P. H. F. *Seleção de Atributos para Estimação de Regiões com Faltas de Alta Impedância*. Dissertação (Trabalho de Conclusão de Curso (Graduação em Engenharia Elétrica)) — Universidade Federal de Minas Gerais, Escola de Engenharia, Belo Horizonte, 2026. Citado 9 vezes nas páginas 3, 7, 8, 9, 10, 11, 13, 20 e 21.

MOLOI, K.; JORDAAN, J. A.; HAMAM, Y. High impedance fault classification and localization method for power distribution network. In: *2018 IEEE PES/IAS PowerAfrica*. [S.l.: s.n.], 2018. p. 84–89. Citado na página 7.

NORDBY, J. *emlearn: Machine Learning inference engine for Microcontrollers and Embedded Devices*. 2019. Zenodo. Disponível em: <https://github.com/emlearn/emlearn>. Citado 2 vezes nas páginas 5 e 10.

PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. Citado 2 vezes nas páginas 5 e 10.

RAY, P. P. A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University – Computer and Information Sciences*, v. 34, n. 4, p. 1595–1623, 2022. Citado na página 5.

SARANGI, S.; SAHU, B. K.; ROUT, P. K. High-impedance fault identification and location by using mode decomposition integrated adaptive multi-kernel extreme learning machine technique for distributed generator-based microgrid. In: *Electrical Engineering*. [S.l.: s.n.], 2023. v. 105, n. 383, p. 406. Citado na página 6.

STMICROELECTRONICS. *RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs*. 2026. Disponível em: <https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf>. Acesso em: 1 nov. 2026. Citado na página 6.

STOCKWELL, R. G.; MANSINHA, L.; LOWE, R. P. Localization of the complex spectrum: the S transform. *IEEE Transactions on Signal Processing*, v. 44, n. 4, p. 998–1001, 1996. Citado na página 11.

WARDEN, P.; SITUNAYAKE, D. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol: O'Reilly Media, 2019. Citado na página 5.